

১.৫৮-বিট LLM: কৃত্রিম বুদ্ধিমত্তার নতুন দিগন্ত

BitNet b1.58 রিসার্চ পেপারের উপর ভিত্তি করে রচিত একটি সম্পূর্ণ বাংলা টেকনিক্যাল গাইড —
Quantization থেকে Bit-packing পর্যন্ত সব কিছু

 BitNet b1.58

 Ternary Weights

 Deep Technical

 সম্পূর্ণ বাংলায়

বিষয়সূচি

- পটভূমি: LLM Quantization কী এবং কেন দরকার?
- ১-বিট LLM কী? — BitNet পরিচিতি
- কেন ১.৫৮ বিট? — Information Theory ব্যাখ্যা
- Ternary Weights: -১, ০, ১ — বিস্তারিত ব্যাখ্যা
- Absolute Mean Quantization — কীভাবে রূপান্তর হয়?
- ৩২-বিট বনাম ১.৫৮-বিট — পাশাপাশি তুলনা
- Large Model-এ Layer-wise Quantization
- Training প্রক্রিয়া: Float থেকে Ternary-তে যাওয়া
- STE (Straight-Through Estimator) — ট্রেনিংয়ের ম্যাজিক
- Bit-packing: মেমরিতে কীভাবে সেভ হয়?
- কম্পিউটার কি Ternary বোঝে? — Binary বনাম Ternary

12. Performance তুলনা: Memory, Latency, Energy
13. BitNet Architecture: nn.Linear থেকে BitLinear
14. সম্পূর্ণ Python কোড উদাহরণ
15. উপসংহার: ভবিষ্যতের AI

১ পটভূমি: LLM Quantization কী এবং কেন দরকার?

সমস্যার গোড়া থেকে শুরু — বড় মডেল চালাতে কেন এত কষ্ট?

আধুনিক Large Language Model (LLM) যেমন LLaMA, GPT ইত্যাদিতে কোটি কোটি **Parameter** বা **Weight** থাকে। উদাহরণস্বরূপ, LLaMA 2-এর ৭ বিলিয়ন প্যারামিটার রয়েছে। এই বিশাল সংখ্যক প্যারামিটার সংরক্ষণ করতে এবং চালাতে প্রচুর RAM, GPU এবং শক্তির প্রয়োজন হয়।



সাধারণত একটি মডেলের প্রতিটি **Weight FP32 (32-bit Floating Point)** ফরম্যাটে থাকে। ৭ বিলিয়ন প্যারামিটারের একটি মডেল লোড করতে প্রায় ২৮ GB মেমরি লাগে — যা সাধারণ মানুষের হাতের নাগালের বাইরে!

Quantization: সমাধানের প্রথম পদক্ষেপ

Quantization হলো এমন একটি প্রক্রিয়া যেখানে মডেলের Weight-গুলোর Precision বা সূক্ষ্মতা ইচ্ছাকৃতভাবে কমিয়ে আনা হয়, যাতে মডেলের সাইজ ছোট হয় এবং কম মেমরিতে চলতে পারে।

❌ সমস্যা (আগে)

- ▶ FP32 বা FP16 ওয়েট
- ▶ বিশাল RAM/GPU দরকার
- ▶ উচ্চ Energy Consumption
- ▶ সাধারণ ডিভাইসে চলে না

✅ সমাধান (Quantization)

- ▶ FP32 → INT8 বা INT4
- ▶ কম মেমরিতে চলে
- ▶ Fine-tuning সম্ভব (LoRA/QLoRA)

► Inference অনেক ধীর

► দ্রুত Inference

► Resource-constrained
ডিভাইসে কাজ

Quantization-এর ধাপগুলো (ঐতিহাসিক ক্রম)

Precision	বিট সংখ্যা	বিশেষত্ব	Memory (7B model)
FP32	32 বিট	সর্বোচ্চ নির্ভুলতা	~28 GB
FP16 / BF16	16 বিট	সাধারণ Training	~14 GB
INT8	8 বিট	LLM.int8() পদ্ধতি	~7 GB
INT4	4 বিট	GGUF, GPTQ পদ্ধতি	~3.5 GB
BitNet 1.58	~1.58 বিট	শুধু -১, ০, +১	~1.4 GB



LoRA ও QLoRA: কম মেমরিতে বড় মডেল Fine-tune করার জন্য ব্যবহৃত পদ্ধতি।

Quantization-এর সাথে মিলিয়ে এই দুটি টেকনিক ব্যবহার করলে সাধারণ GPU-তেও বিশাল মডেল Fine-tune করা যায়।



১-বিট LLM কী? — BitNet পরিচিতি

Quantization-এর চরম সীমা — যেখানে Weight মাত্র তিনটি মান নিতে পারে

গবেষকরা এই রিসার্চ পেপারে **BitNet b1.58** নামে একটি নতুন ধরনের LLM আর্কিটেকচার প্রস্তাব করেছেন। এটি "১-বিট LLM ভ্যারিয়েন্ট" নামেও পরিচিত।



মূল সংজ্ঞা: BitNet b1.58 হলো এমন একটি Transformer-ভিত্তিক LLM যেখানে প্রতিটি Weight বা Parameter শুধুমাত্র তিনটি মানের যেকোনো একটি নিতে পারে: -1 , 0 , বা $+1$ — এবং আর কিছু নয়।

BitNet-এর মূল দাবি

- 1** সমান একই Model Size ও Training Token-এর Full Precision **Performance:** (FP16) মডেলের সমতুল্য Perplexity ও End-to-End Task Performance।
- 2** অনেক কম Latency (গতি), Memory (RAM/GPU), Throughput এবং Energy খরচ: Consumption সব দিক থেকে উল্লেখযোগ্যভাবে সশ্রুয়ী।
- 3** **Feature Filtering:** Weight-এ '০' থাকায় অপ্রাসঙ্গিক তথ্য ফিল্টার করার শক্তি বাড়ে।
- 4** ৩B+ মডেলে **Full Precision Match:** ৩ বিলিয়ন বা তার বেশি প্যারামিটারের মডেলে এটি Full Precision Baseline-এর সমতুল্য কাজ করে।



উপমা

আপনি একটি ভোটের আয়োজন করছেন। প্রতিটি ভোটার শুধু তিনটি সিদ্ধান্ত নিতে পারবে: পক্ষে ($+1$), বিপক্ষে (-1), বা নিরপেক্ষ (0)। এই সরল তিনটি মান দিয়েও একটি সম্পূর্ণ ও অর্থবহ ভোটের ফলাফল পাওয়া সম্ভব — ঠিক যেমন BitNet-এ তিনটি মান দিয়ে পুরো LLM চালানো সম্ভব।



কেন ১.৫৮ বিট? — Information Theory ব্যাখ্যা

৩টি মান থাকলে গাণিতিকভাবে ঠিক কতটুকু তথ্য সংরক্ষিত হয়?

বিট (Bit) কী?

কম্পিউটার বিজ্ঞানে ১ বিট দিয়ে সর্বোচ্চ ২টি মান প্রকাশ করা যায়: ০ এবং ১।

২ বিট দিয়ে সর্বোচ্চ ৪টি মান: ০০, ০১, ১০, ১১।

সাধারণ সূত্র: n বিট দিয়ে 2^n টি মান প্রকাশ করা যায়।

৩টি মানের জন্য কত বিট দরকার?

ঠিক ৩টি মান প্রকাশ করতে কতটুকু তথ্যের প্রয়োজন তা নির্ণয় করা হয় **Binary Logarithm ($\log_2$)** দিয়ে:

$$\log_2(3) \approx 1.58496 \text{ বিট}$$

এই গাণিতিক হিসাবের ভিত্তিতেই মডেলটির নাম দেওয়া হয়েছে **BitNet b1.58**।

সম্ভাব্য মানের সংখ্যা	উদাহরণ	গাণিতিক বিট ($\log_2$)	বাস্তব বিট (হার্ডওয়্যার)
২টি মান	০ বা ১	1.00 বিট	1 বিট
৩টি মান	-১, ০, +১	1.58 বিট	2 বিট (Naïve) বা ~1.6 বিট (Packed)
৪টি মান	0,1,2,3	2.00 বিট	2 বিট
256টি মান	INT8	8.00 বিট	8 বিট



গুরুত্বপূর্ণ পার্থক্য: "১.৫৮ বিট" হলো তাত্ত্বিক তথ্য-ঘনত্ব (Information Theory)-র মাপ।
বাস্তবে হার্ডওয়্যারে এটি সংরক্ষণ করতে Bit-packing পদ্ধতি ব্যবহার করা হয় (পরে বিস্তারিত আলোচনা করা হবে)।

৪

Ternary Weights: -১, ০, +১ — বিস্তারিত ব্যাখ্যা

তিনটি মান দিয়ে কীভাবে একটি পূর্ণ Neural Network কাজ করে?

সাধারণ Neural Network-এ কী হয়?

একটি সাধারণ Transformer-এ Forward Propagation-এর মূল গাণিতিক সমীকরণ:

$$\text{Output} = \sum (W_i \times X_i) + B$$

এখানে W_i হলো Weight, X_i হলো Input, এবং B হলো Bias। এই সমীকরণে প্রতিটি ধাপে গুণ (Multiplication) এবং যোগ (Addition) — দুটোই করতে হয়।

BitNet-এ কী পরিবর্তন হয়?

যখন W_i -এর মান শুধুমাত্র -১, ০, বা +১ হয়, তখন গুণের ধাপটি প্রায় অর্থহীন হয়ে পড়ে:

Weight (W)	Input (X) × Weight	কার্যকর অপারেশন
+১	$X \times (+১) = X$	শুধু যোগ (+X)
-১	$X \times (-১) = -X$	শুধু বিয়োগ (-X)
০	$X \times ০ = ০$	কিছুই করতে হয় না



বিপ্লবী পরিবর্তন: জটিল Floating-point Multiplication সম্পূর্ণ বাদ দিয়ে শুধু Integer Addition করলেই হয়! GPU-তে Multiplication অনেক বেশি সময় ও বিদ্যুৎ নেয়। এই পরিবর্তনে গণনার খরচ বিশাল পরিমাণে কমে যায়।



উপমা

ভাবুন আপনি একটি দোকানে হিসাব করছেন। FP32 পদ্ধতিতে হিসাব করলে প্রতিটি জিনিসের জন্য ক্যালকুলেটর বের করে দশমিকসহ গুণ করতে হবে। BitNet পদ্ধতিতে প্রতিটি জিনিস হয় "যোগ করো (+১)", "বাদ দাও (-১)" বা "ধর্তব্যের মধ্যে নাই (০)" — শুধু মাথায় মাথায় গণনা করলেই চলে!

Feature Filtering: '০' ওয়েটের বিশেষ ভূমিকা

BitNet-এর গবেষকরা উল্লেখ করেছেন যে ওয়েটে '০' থাকার কারণে **Feature Filtering** সম্ভব হয়। যখন একটি Weight ০ হয়, তখন সেই নির্দিষ্ট Input-এর প্রভাব সম্পূর্ণ বাদ দেওয়া হয় — অর্থাৎ মডেল নিজে নিজে অপ্রাসঙ্গিক তথ্য ফিল্টার করতে পারে। এটি মডেলের Modeling Capacity বা বোঝার শক্তি আরও শক্তিশালী করে।



Absolute Mean Quantization — কীভাবে রূপান্তর হয়?

FP32 থেকে Ternary (-১, ০, +১) রূপান্তরের সম্পূর্ণ গাণিতিক প্রক্রিয়া

সাধারণ 32-বিট Weight-গুলোকে -১, ০, +১-এ রূপান্তর করতে **Absolute Mean Quantization** নামের একটি গাণিতিক পদ্ধতি ব্যবহার করা হয়। এই প্রক্রিয়া মূলত ৩টি ধাপে সম্পন্ন হয়।

উদাহরণ: ৪টি **Weight** নিয়ে কাজ করা যাক

ধরুন আমাদের মডেলের ৪টি Weight আছে:



উদাহরণ ডেটা

মূল FP32 ওয়েট $W = [0.5, -1.2, 0.1, 0.8]$

1

ধাপ ১: **Threshold (y)** বের করা

সকল Weight-এর Absolute Value (চিহ্ন বাদ দিয়ে) নিয়ে গড় (Mean) বের করা হয়:

1 2
3 4ধাপ ১: **Absolute Mean**
 $|W| = [0.5, 1.2, 0.1, 0.8]$ $y = (0.5 + 1.2 + 0.1 + 0.8) / 4$ $y = 2.6 / 4$ $y = 0.65$ # এটাই আমাদের *Threshold* বা *Scaling Factor*

2

ধাপ ২: **Scaling** — প্রতিটি Weight-কে y দিয়ে ভাগ করা1 2
3 4ধাপ ২: **Scaling**
 $Scaled = W / y$ $Scaled[0] = 0.5 / 0.65 \approx +0.77$ $Scaled[1] = -1.2 / 0.65 \approx -1.84$ $Scaled[2] = 0.1 / 0.65 \approx +0.15$ $Scaled[3] = 0.8 / 0.65 \approx +1.23$

3

ধাপ ৩: **Rounding + Clipping** — সর্বোচ্চ ও সর্বনিম্ন -১ এবং +১-এ আটকে দেওয়া1 2
3 4ধাপ ৩: **Round & Clip**
 $Quantized[0] = \text{round}(0.77) = 1 \rightarrow \text{clip}(-1, 1) \rightarrow +1$ $Quantized[1] = \text{round}(-1.84) = -2 \rightarrow \text{clip}(-1, 1) \rightarrow -1$ # ক্লিপ হয়েছে গেছে $Quantized[2] = \text{round}(0.15) = 0 \rightarrow \text{clip}(-1, 1) \rightarrow 0$ $Quantized[3] = \text{round}(1.23) = 1 \rightarrow \text{clip}(-1, 1) \rightarrow +1$ # চূড়ান্ত ফলাফল: # মূল $[0.5, -1.2, 0.1, 0.8]$ # Ternary $[+1, -1, 0, +1]$


Clipping কেন দরকার? ভাগ করার পর যদি কোনো মান +১-এর বেশি বা -১-এর কম হয়ে যায় (যেমন -১.৮৪ বা +১.২৩), তাহলে তাকে জোর করে -১ বা +১-এ আটকে দেওয়া হয়। এই প্রক্রিয়াকে **Clipping** বলে।

$$Q(W) = \text{clip}(\text{round}(W / \gamma), -1, +1)$$

যেখানে $\gamma = \text{mean}(|W|)$

৬ ৩২-বিট বনাম ১.৫৮-বিট — পাশাপাশি তুলনা

একটি নির্দিষ্ট Weight মান উভয় পদ্ধতিতে কেমন দেখায়?

ধরুন একটি Weight-এর মান হলো **0.15625**। এই মানটি দুটি ভিন্ন মডেলে কীভাবে সংরক্ষিত হয়:

৩২-বিট **FP32** রূপ

 **FP32** মেমরি রিপ্রেজেন্টেশন # 0.15625 → FP32 Binary (IEEE 754) # 
Sign (1 bit) | Exponent (8 bits) | Mantissa (23 bits)
0 | 01111100 | 01000000000000000000000 # মোট: ৩২টি
বিট = ৪ বাইট শুধুমাত্র একটি Weight-এর জন্য!

১.৫৮-বিট **Ternary** রূপ

 **BitNet Ternary** রিপ্রেজেন্টেশন # 0.15625 → Quantization → শূন্যের কাছাকাছি →
০ হয়ে যায় ০ # একটি মাত্র মান! (-১, ০, বা +১) # মোট: তাত্ত্বিকভাবে মাত্র ১.৫৮ বিট একটি
Weight-এর জন্য

বৈশিষ্ট্য	৩২-বিট (FP32)	১৬-বিট (FP16)	১.৫৮-বিট (BitNet)
একটি Weight-এর গড় মেমরি	32 বিট	16 বিট	~1.6 বিট
৫টি Weight-এর মেমরি	160 বিট	80 বিট	8 বিট (1 Byte)
গণনার প্রকৃতি	Multiply + Add	Multiply + Add	শুধু Add/Subtract
GPU প্রয়োজনীয়তা	অনেক বেশি	বেশি	অনেক কম
মান প্রকাশের সক্ষমতা	~4 বিলিয়ন মান	~65,000 মান	মাত্র ৩টি মান



সাশ্রয়ের হিসাব: ৩২-বিটের তুলনায় BitNet মডেলে প্রতিটি Weight গড়ে ২০ গুণ কম জায়গা নেয়। একটি 7B প্যারামিটারের মডেল যদি FP32-এ 28 GB নেয়, তবে BitNet-এ তা মাত্র ~1.4 GB-এ নেমে আসে!

৭ Large Model-এ Layer-wise Quantization

৫০ বিলিয়ন প্যারামিটারের মডেলে কি একসাথে সব ওয়েটের গড় বের হয়?



ভুল ধারণা: "৫০ বিলিয়ন প্যারামিটারের মডেল কি সব ওয়েটের একসাথে গড় বের করে?" — না। এটা করলে মডেল বোকা হয়ে যাবে।

বাস্তবে দুটি স্তরে Quantization করা হয়:

✓ পদ্ধতি ১: **Layer-wise (Tensor-wise)**

প্রতিটি Transformer Layer আলাদাভাবে Quantize করা হয়। একটি Layer-এ হয়তো ১০ লক্ষ Weight আছে — সে শুধু সেই ১০ লক্ষের গড় বের করে।

✓ পদ্ধতি ২: **Group-wise**

আরও নিখুঁত করতে প্রতিটি Layer-কেও ছোট ছোট ব্লকে (৬৪ বা ১২৮টি Weight-এর গ্রুপ) ভাগ করে আলাদা Threshold ব্যবহার করা হয়।

💡 উপমা

একটি দেশের সবার আয়ের গড় বের করলে যা পাওয়া যায় তা দিয়ে সাধারণ মানুষের অর্থনীতি বোঝা যায় না — কারণ কোটিপতির আয় গড়কে বিকৃত করে দেয়। কিন্তু পেশা অনুযায়ী বা এলাকা অনুযায়ী আলাদা আলাদা গড় বের করলে ছবিটা অনেক সঠিক হয়। Layer-wise Quantization-এ ঠিক এটাই করা হয়।



Python: Layer-wise Quantization ধারণা

```
import numpy as np

def absolute_mean_quantize(weight_matrix):
    """ একটি সম্পূর্ণ Weight Matrix (Layer)-কে Quantize করে। পুরো মডেলের Weight নয়, শুধু এই একটি Layer-এর Weight। """
    # ধাপ ১: এই Layer-এর সব Weight-এর Absolute Mean বের করা
    gamma = np.mean(np.abs(weight_matrix))
    # ধাপ ২: Scaling
    scaled = weight_matrix / (gamma + 1e-8)
    # 1e-8 দেওয়া হয় zero division এড়াতে
    # ধাপ ৩: Round + Clip → Ternary
    quantized = np.clip(np.round(scaled), -1, 1).astype(np.int8)
    return quantized, gamma

# gamma পরে Inference-এ দরকার হবে
# উদাহরণ
layer_weights = np.array([[0.5, -1.2, 0.1], [0.8, -0.3, 1.5]])
q_weights, gamma = absolute_mean_quantize(layer_weights)

print(f"Threshold (γ): {gamma:.4f}")
print(f"Ternary Weights:\n{q_weights}")

# আউটপুট:
# Threshold (γ): 0.7333
# Ternary Weights:
# [[ 1 -2  0]
#  [ 1  0  2]]
# ক্লিপের পর: [1, -1, 0] # [ 1  0  2]] → ক্লিপের পর: [1, 0, 1]
```



Training প্রক্রিয়া: Float থেকে Ternary-তে যাওয়া

কোডে Weight Float দেখায়, কিন্তু মডেল Ternary-তে কাজ করে — এর রহস্য কী?

BitNet মডেল Train করার সময় একটি চমৎকার দ্বৈত কৌশল ব্যবহার করা হয়। মডেল দুটি সমান্তরাল লেভেলে কাজ করে।

দুই স্তরের Weight ব্যবস্থাপনা



Master Weights (লুকানো)

FP16/FP32 ফরম্যাটে। এগুলো দিয়ে ধীরে ধীরে শেখা হয়। Backpropagation এখানে আপডেট করে। Training শেষে মুছে ফেলা হয়।



Quantized Weights (ব্যবহৃত)

Ternary (-১, ০, +১)। Forward Pass-এ হিসাব এখানে হয়। Master Weight থেকে তৈরি। প্রতি Forward Pass-এ নতুন করে তৈরি হয়।

Training-এর সম্পূর্ণ চক্র

1

Initialization: শুরুতে সব Master Weight র্যান্ডম ছোট FP32 ভগ্নাংশ (যেমন: 0.0125, -0.0043) হিসেবে তৈরি হয়।

2

Forward Pass: Master Weight থেকে Absolute Mean Quantization করে Ternary Weight তৈরি। এই Ternary Weight দিয়ে Addition-only গণনা করে Output বের করা।

3

Loss Calculation: Output-এর সাথে সঠিক উত্তরের পার্থক্য (Loss) নির্ণয়।

4

Backpropagation + STE: Loss থেকে Gradient বের করে Master Weight আপডেট করা (STE পদ্ধতিতে — পরের সেকশনে বিস্তারিত)।

5

পুনরাবৃত্তি: লক্ষ লক্ষ বার এই চক্র চলে। Master Weight ধীরে ধীরে উন্নত হয়।

6

Training

শেষ:

Master Weight থেকে চূড়ান্ত Ternary বের করে মডেল ফাইলে সেভ। FP32

Master Weight মুছে ফেলা হয়।

৯

STE (Straight-Through Estimator) — ট্রেনিংয়ের

ম্যাজিক

Quantization ফাংশনে কোনো Derivative নেই — তাহলে Learning কীভাবে হয়?

সমস্যাটা কী?

Backpropagation-এর জন্য প্রতিটি Function-এর **Derivative (Gradient)** দরকার। কিন্তু Rounding এবং Clipping Function-এর Derivative সব জায়গায় শূন্য বা অসংজ্ঞায়িত — তাই সাধারণ পদ্ধতিতে Quantization-এর মধ্য দিয়ে Learning সম্ভব নয়।



মূল সমস্যা: $\text{round}(-1.84) = -2 \rightarrow \text{clip}(-1, 1) = -1$ এই ফাংশনের Derivative সংজ্ঞায়িত নয়, ফলে Gradient প্রবাহিত হতে পারে না। Gradient ছাড়া Weight আপডেট করা সম্ভব নয়।

STE সমাধান

Straight-Through Estimator (STE) একটি সরল কিন্তু কার্যকর কৌশল: **Backward Pass-এ Quantization** ফাংশনকে সম্পূর্ণ উপেক্ষা করা হয় এবং Gradient সরাসরি Master Weight-এ পাঠিয়ে দেওয়া হয়।

**Python: STE Implementation (PyTorch)**

```
import torch import
torch.nn as nn class
TernaryQuantize(torch.autograd.Function): """ STE-ভিত্তিক Ternary
Quantization Function। Forward: সঠিকভাবে Quantize করে। Backward:
Gradient সরাসরি পাস করে (Quantization উপেক্ষা)। """ @staticmethod def
forward(ctx, weight): # Absolute Mean Quantization gamma =
```

```

weight.abs().mean() scaled = weight / (gamma + 1e-8)
quantized = scaled.round().clamp(-1, 1)
ctx.save_for_backward(weight) # Backward-এর জন্য রাখা return
quantized @staticmethod def backward(ctx, grad_output): #
STE: Gradient হুবহু পাস করে দাও, Quantization উপেক্ষা করো # এটিই STE-এর মূল
কৌশল — 'Straight Through' করা return grad_output # grad_output
পরিবর্তন ছাড়াই ফেরত দেওয়া class BitLinear(nn.Module): """nn.Linear-এর
Ternary সংস্করণ""" def __init__(self, in_features,
out_features): super().__init__() # Master Weight: FP32 হিসেবে
ওর self.weight = nn.Parameter( torch.randn(out_features,
in_features) * 0.02 ) def forward(self, x): # Forward Pass:
Ternary Weight ব্যবহার করো ternary_weight =
TernaryQuantize.apply(self.weight) # শুধু Addition
(Multiplication-free linear transformation) return
nn.functional.linear(x, ternary_weight) # Backward Pass-এ
PyTorch স্বয়ংক্রিয়ভাবে STE ব্যবহার করে # self.weight (Master Weight) আপডেট
হয় # ---- ব্যবহার ---- layer = BitLinear(512, 256) x =
torch.randn(4, 512) # Batch of 4 samples output = layer(x)
loss = output.sum() loss.backward() # STE কাজ করে: Master
Weight-এ Gradient যায় print(f"Master Weight Gradient exists:
{layer.weight.grad is not None}") # Output: Master Weight
Gradient exists: True

```

💡 উপমা

মনে করুন আপনি একটি গোলাকার সিঁড়ি দিয়ে উঠছেন। প্রতিটি ধাপে ওঠার সময় (Forward) আপনাকে সঠিকভাবে প্রতিটি ধাপ মাড়াতে হয়। কিন্তু নামার সময় (Backward) — অর্থাৎ ভুল শুধরানোর সময় — আপনি সিঁড়ির ধাপগুলো উপেক্ষা করে সরাসরি নিচে নামেন। STE ঠিক এই কাজটিই করে।

১০ Bit-packing: মেমরিতে কীভাবে সেভ হয়?

Ternary মান বাইনারি ফাইলে কীভাবে চলে যায়?

Training শেষে মডেল ফাইলে (.bin বা .safetensors) সেভ করার সময় একটি চমৎকার **Bit-packing** কৌশল ব্যবহার করা হয়।

গণিতের চমক

৫টি Ternary মান (-১, ০, +১) একসাথে নিলে মোট সম্ভাব্য সমন্বয়ের সংখ্যা:

$$3^5 = 243 \text{টি ভিন্ন সমন্বয়}$$

একটি ৮-বিট **Integer (INT8)** সর্বোচ্চ ২৫৬টি ভিন্ন মান (0 থেকে 255) ধারণ করতে পারে।



$243 < 256 \rightarrow$ তাই ৫টি Ternary Weight-কে একটিমাত্র INT8 ব্লকে প্যাক করা সম্ভব!

প্রতিটি Weight-এর গড় মেমরি: $8 \div 5 = 1.6$ বিট — যা তাত্ত্বিক 1.58 বিটের প্রায় সমান।



Python: Bit-packing ও Unpacking

```
import numpy as np

def ternary_to_int8(ternary_weights): """ Ternary Weight গুলোকে (-1, 0, 1) থেকে (0, 1, 2) তে map করে তারপর Base-3 এনকোডিংয়ে একটি INT8-এ প্যাক করে। """ # (-1, 0, 1) → (0, 1, 2): কারণ নেগেটিভ সংখ্যা সরাসরি base-3-এ রাখা যায় না
mapped = ternary_weights + 1 # -1→0, 0→1, 1→2
packed_values = []
for i in range(0, len(mapped), 5):
    chunk = mapped[i:i+5] # Base-3 এনকোডিং: d0*3^0 + d1*3^1 + d2*3^2 + d3*3^3 + d4*3^4
    packed = sum(chunk[j] * (3**j) for j in range(len(chunk)))
    packed_values.append(packed)
return np.array(packed_values, dtype=np.uint8)

def int8_to_ternary(packed): """ INT8 থেকে Ternary Weight পুনরুদ্ধার করা (Unpacking) """
ternary = []
for val in packed:
    chunk = []
    for _ in range(5):
        chunk.append((val % 3) - 1) # (0, 1, 2) → (-1, 0, 1)
    val //= 3
    ternary.extend(chunk)
return np.array(ternary, dtype=np.int8)

# ----- পরীক্ষা করা যাক -----
original = np.array([1, -1, 0, 1, 0], dtype=np.int8)
print(f"মূল Ternary: {original} → মেমরি: {original.nbytes} বাইট (Naïve)")
packed = ternary_to_int8(original)
print(f"Packed INT8: {packed} → মেমরি: {packed.nbytes} বাইট")
unpacked = int8_to_ternary(packed[:len(original)])
print(f"পুনরুদ্ধার: {unpacked}")
print(f"সঠিক মিল: {np.array_equal(original, unpacked)}")
# আউটপুট: # মূল Ternary: [ 1 -1 0 1 0] → মেমরি: 5 বাইট
```

```
(Naïve) # Packed INT8: [17] → মেমরি: 1 বাইট ← ৫ গুণ কম! # পুনরুদ্ধার: [ 1  
-1 0 1 0] # সঠিক মিল: True
```

🔑 Bit-packing-এর সুবিধা

- ৫টি Weight → ১টি INT8 ব্লক → প্রতি Weight গড়ে মাত্র ১.৬ বিট মেমরি
- FP32-এর তুলনায় ~২০ গুণ কম মেমরি ব্যবহার
- FP16-এর তুলনায় ~১০ গুণ কম মেমরি ব্যবহার
- Inference-এর সময় দ্রুত Unpack করে কাজ করা যায়

১১

কম্পিউটার কি Ternary বোঝে? — Binary বনাম Ternary Hardware

সিলিকন ট্রানজিস্টরের দুনিয়ায় তৃতীয় অবস্থা নেই

কম্পিউটার কেন শুধু বাইনারি বোঝে?

আধুনিক কম্পিউটারের ভেতরে কোটি কোটি **Transistor** আছে, যা মূলত বৈদ্যুতিক সুইচ। এই সুইচের ফিজিক্যাল অবস্থা মাত্র দুটি হতে পারে:

1 (ভোল্টেজ আছে = ON)

উচ্চ বৈদ্যুতিক ভোল্টেজ → Transistor
চালু → বাইনারি ১

0 (ভোল্টেজ নেই = OFF)

শূন্য বৈদ্যুতিক ভোল্টেজ → Transistor বন্ধ
→ বাইনারি ০

একটি তৃতীয় অবস্থা (যেমন "অর্ধেক ভোল্টেজ") তৈরি করা এবং নির্ভরযোগ্যভাবে পার্থক্য করা অত্যন্ত কঠিন। তাই সমস্ত আধুনিক কম্পিউটার হার্ডওয়্যার সম্পূর্ণ বাইনারি পদ্ধতিতে কাজ করে।

তাহলে Ternary কীভাবে কম্পিউটারে কাজ করে?

Software এবং Encoding-এর সাহায্যে! Bit-packing পদ্ধতিতে Ternary মানগুলোকে বাইনারি কোডে রূপান্তর করে সংরক্ষণ করা হয়। Inference-এর সময় সেটা Decode করে ব্যবহার করা হয়।



ইতিহাসের একটি মজার ঘটনা: ১৯৫৮ সালে সোভিয়েত ইউনিয়নে মস্কো স্টেট ইউনিভার্সিটির বিজ্ঞানীরা "Setun" নামে একটি সত্যিকারের Ternary কম্পিউটার তৈরি করেছিলেন — যা হার্ডওয়্যার লেভেলে সরাসরি ৩টি অবস্থা বুঝত। কিন্তু বাইনারি হার্ডওয়্যার সস্তা ও নির্ভরযোগ্য হওয়ায় Ternary কম্পিউটার এগোতে পারেনি।

ভবিষ্যতের সম্ভাবনা: নতুন Hardware

BitNet রিসার্চ পেপারে উল্লেখ করা হয়েছে যে ১.৫৮-বিট মডেলের নতুন Computing Paradigm নতুন Hardware ডিজাইনের সুযোগ তৈরি করেছে। ভবিষ্যতে Ternary-optimized Chip তৈরি হলে আরও বেশি সাশ্রয় সম্ভব।

বিষয়	সাধারণ Binary Computer	Ternary BitNet
Hardware Level	সরাসরি 0 ও 1	0 ও 1 দিয়ে Emulate
Software Level	FP32/FP16 Weight	-1, 0, +1 এনকোড করা
File Storage	Raw Binary Bytes	Bit-packed Binary Bytes
Computation	Float Multiply + Add	শুধু Integer Add



Performance তুলনা: Memory, Latency, Energy

রিসার্চ পেপারের পরিমাপ থেকে বাস্তব পার্থক্য

~20×

FP32 থেকে কম Memory
ব্যবহার

~10×

FP16 থেকে কম Memory
ব্যবহার

↓↓↓

Latency ও Energy
উল্লেখযোগ্যভাবে কমেছে

মডেল সাইজ বনাম Latency

রিসার্চ পেপারে LLaMA (নীল) ও BitNet (কমলা) মডেলের তুলনায় দেখানো হয়েছে যে একই প্যারামিটার সংখ্যায় BitNet অনেক কম Latency-তে কাজ করে। মডেলের সাইজ যত বাড়ে, পার্থক্য তত বেশি স্পষ্ট হয়।

মডেল	Parameter	Memory (BitNet vs LLaMA)	PPL (Perplexity)
LLaMA 700M	700M	2.08 GB	12.33
BitNet 700M	700M	1.18 GB (↓43%)	8.96
LLaMA 1.3B	1.3B	1.14 GB	11.29
BitNet 1.3B	1.3B	0.97 GB (↓15%)	তুলনীয়



Perplexity (PPL): এটি LLM-এর কার্যকারিতা মাপার একটি মেট্রিক। সংখ্যা যত কম, মডেল তত ভালো। BitNet-এর PPL সমতুল্য FP16 মডেলের কাছাকাছি — অর্থাৎ ছোট হয়েও সমান ভালো পারফর্ম করে।

Energy Consumption

Addition অপারেশন Multiplication-এর তুলনায় অনেক কম শক্তি খরচ করে। যেহেতু অধিকাংশ Chip-এর Performance সীমা শক্তি-ব্যয় দ্বারা নির্ধারিত হয়, তাই শক্তি সাশ্রয় মানেই দ্রুততর গণনা।

১৩

BitNet Architecture: nn.Linear থেকে BitLinear

Transformer আর্কিটেকচারে ঠিক কোন জায়গায় পরিবর্তন আসে?

BitNet-এর Transformer আর্কিটেকচারে মূলত একটাই পরিবর্তন: `nn.Linear` লেয়ারকে

`BitLinear` দিয়ে প্রতিস্থাপন করা হয়।

```
Python: সম্পূর্ণ BitLinear Layer Implementation import torch import
torch.nn as nn import torch.nn.functional as F class
TernaryQuantizeSTE(torch.autograd.Function): """STE-ভিত্তিক
Quantization""" @staticmethod def forward(ctx, weight): gamma
= weight.abs().mean() + 1e-8 return (weight /
gamma).round().clamp(-1, 1) @staticmethod def backward(ctx,
grad_output): return grad_output # STE: সরাসরি পাস class
BitLinear(nn.Module): """ BitNet-এর BitLinear Layer। Training:
1.58-বিট Weight + 8-বিট Activation Inference: Bit-packed Ternary
Weight """ def __init__(self, in_features, out_features,
bias=True): super().__init__() self.in_features = in_features
self.out_features = out_features # Master Weight (FP32) -
Training-এ ব্যবহৃত self.weight =
nn.Parameter( torch.empty(out_features,
in_features).normal_(0, 0.02) ) if bias: self.bias =
nn.Parameter(torch.zeros(out_features)) else:
self.register_parameter('bias', None) def forward(self, x): #
Weight Quantization (STE সহ) ternary_w =
TernaryQuantizeSTE.apply(self.weight) # Input Activation
Quantization (8-bit) # (সরলতার জন্য এখানে বাদ দেওয়া হলো) return
F.linear(x, ternary_w, self.bias) def extra_repr(self):
return f"in={self.in_features}, out={self.out_features},
```

```

type=BitLinear(1.58-bit)" # — সাধারণ Transformer ব্লক: nn.Linear
→ BitLinear — class SimpleTransformerBlock(nn.Module): def
__init__(self, d_model=512, d_ff=2048): super().__init__() #
সাধারণ Transformer: nn.Linear ব্যবহার করে # self.linear1 =
nn.Linear(d_model, d_ff) ← আগের কোড # self.linear2 =
nn.Linear(d_ff, d_model) ← আগের কোড # BitNet: BitLinear দিয়ে
প্রতিস্থাপন self.linear1 = BitLinear(d_model, d_ff) # ← নতুন
self.linear2 = BitLinear(d_ff, d_model) # ← নতুন self.norm =
nn.LayerNorm(d_model) def forward(self, x): residual = x x =
self.norm(x) x = torch.relu(self.linear1(x)) x =
self.linear2(x) return x + residual # ---- পরীক্ষা ---- block =
SimpleTransformerBlock() x = torch.randn(2, 10, 512) #
[batch=2, seq=10, dim=512] output = block(x) print(f"Input
shape: {x.shape}") print(f"Output shape: {output.shape}")
print(f"Block structure:\n{block}") # Output shape:
torch.Size([2, 10, 512])

```



Training Configuration: BitNet b1.58 মডেলকে শুরু থেকে (from scratch)

ট্রেন করা হয় — **1.58-বিট Weight** এবং **8-বিট Activation** নিয়ে। এটি Post-training Quantization নয়; এটি Training-time Quantization।

১৪ সম্পূর্ণ Python কোড উদাহরণ

Quantization থেকে Bit-packing পর্যন্ত — একটি সম্পূর্ণ runnable উদাহরণ



Python: সম্পূর্ণ BitNet Demonstration

```

""" BitNet b1.58 সম্পূর্ণ
Demonstration ===== এই কোড চালাতে: pip
install numpy torch """ import numpy as np import torch
import torch.nn as nn #
===== # ১. Absolute Mean
Quantization (NumPy) # =====

```

```

def quantize_weights(W): """FP32 Weight → Ternary (-1, 0,
+1)""" gamma = np.mean(np.abs(W)) + 1e-8 scaled = W / gamma
return np.clip(np.round(scaled), -1, 1).astype(np.int8),
gamma # ===== # ২. Bit-
packing: Ternary → INT8 #
===== def
pack_ternary(ternary: np.ndarray) -> np.ndarray: """Ternary
Array → Packed INT8 Array""" mapped =
ternary.astype(np.int32) + 1 # -1→0, 0→1, 1→2 n_full =
len(mapped) // 5 packed = [] for i in range(n_full): chunk =
mapped[i*5:(i+1)*5] val = int(sum(chunk[j] * (3**j) for j in
range(5))) packed.append(val) if len(mapped) % 5 != 0: # শেষ
অসম্পূর্ণ chunk = mapped[n_full*5:] val = int(sum(chunk[j] *
(3**j) for j in range(len(chunk)))) packed.append(val) return
np.array(packed, dtype=np.uint8) def unpack_ternary(packed:
np.ndarray, original_len: int) -> np.ndarray: """Packed INT8
→ Ternary Array""" result = [] for val in packed: chunk = []
for _ in range(5): chunk.append((int(val) % 3) - 1) val //= 3
result.extend(chunk) return np.array(result[:original_len],
dtype=np.int8) # ===== # ৩.
Ternary Matrix Multiplication #
===== def ternary_matmul(x:
np.ndarray, W_ternary: np.ndarray) -> np.ndarray: """ Ternary
Weight দিয়ে Matrix Multiplication | কার্যত শুধু Addition/Subtraction |
""" result = np.zeros((x.shape[0], W_ternary.shape[0])) for
i, row in enumerate(W_ternary): pos_mask = (row == 1)
neg_mask = (row == -1) # +1 ওয়েট: যোগ করো | -1 ওয়েট: বিয়োগ করো | 0
ওয়েট: কিছু করো না result[:, i] = (x[:, pos_mask].sum(axis=1) - x[:,
neg_mask].sum(axis=1)) return result #
===== # ৪. মেমরি সাশ্রয়ের হিসাব #
===== def
memory_comparison(n_params: int): fp32_bytes = n_params * 4
fp16_bytes = n_params * 2 bitnet_bytes = np.ceil(n_params /
5) # Bit-packed print("\n📊 মেমরি তুলনা") print(f"{'পদ্ধতি':15}
{'মেমরি':>12} {'সাশ্রয়':>10}") print("-" * 40) print(f"{'FP32':15}
{fp32_bytes/1e9:>10.2f} GB {'-':>10}") print(f"{'FP16':15}
{fp16_bytes/1e9:>10.2f} GB {fp32_bytes/fp16_bytes:>9.1f}x")
print(f"{'BitNet 1.58':15} {bitnet_bytes/1e9:>10.3f} GB
{fp32_bytes/bitnet_bytes:>9.1f}x") #
===== # ৫. সব কিছু একসাথে চালানো #
===== if __name__ ==
 "__main__": print("=" * 50) print(" BitNet b1.58 — সম্পূর্ণ প্রদর্শনী")

```

```
print("=" * 50) np.random.seed(42) # উদাহরণ Weight Matrix (৪x৬)
W_fp32 = np.random.randn(4, 6).astype(np.float32) print("\n🔵
মূল FP32 Weight (৪x৬):\n", np.round(W_fp32, 3)) # Quantization
W_ternary, gamma = quantize_weights(W_fp32) print(f"\n🟢
Ternary Weight (γ={gamma:.4f}):\n", W_ternary) # Bit-packing
flat = W_ternary.flatten() packed = pack_ternary(flat)
unpacked = unpack_ternary(packed,
len(flat)).reshape(W_ternary.shape) print(f"\n📦 Bit-
packing:") print(f" মূল ({flat.nbytes} বাইট) → Packed
({packed.nbytes} বাইট)") print(f" সঠিক পুনরুদ্ধার:
{np.array_equal(W_ternary, unpacked)}") # Ternary Matrix
Multiplication X = np.random.randn(3, 6).astype(np.float32)
output = ternary_matmul(X, W_ternary) print(f"\n⚡ Ternary
MatMul (Addition-only:") print(f" Input: {X.shape} → Output:
{output.shape}") # মেমরি তুলনা (7B মডেল)
memory_comparison(7_000_000_000) print(f"\n✅ প্রদর্শনী সম্পন্ন!")
```



প্রত্যাশিত আউটপুট (মেমরি তুলনা):

FP32 → ~28.00 GB | FP16 → ~14.00 GB | BitNet 1.58 → ~1.40 GB
(20x সাশ্রয়)

১৫ উপসংহার: ভবিষ্যতের AI

১.৫৮-বিট LLM কীভাবে AI-এর ভবিষ্যৎ বদলে দিতে পারে

🔑 এই গাইডে আমরা যা শিখলাম

- **Quantization:** বড় মডেলকে ছোট করার প্রক্রিয়া — FP32 → FP16 → INT8 → 1.58-bit
- **BitNet b1.58:** Weight শুধু -১, ০, +১ — Ternary পদ্ধতিতে কাজ করে
- ১.৫৮ বিট কেন: $\log_2(3) \approx 1.58$ — তিনটি মানের Information Entropy

- **Absolute Mean Quantization:** গড় (y) দিয়ে ভাগ করে Round + Clip করা
- **Layer-wise** পদ্ধতি: পুরো মডেল নয়, প্রতিটি Layer আলাদাভাবে Quantize
- **Master Weights + STE:** Training-এ FP32 রেখে Forward-এ Ternary ব্যবহার
- **Bit-packing:** ৫ Ternary $\rightarrow$ ১ INT8 $\rightarrow$ প্রতি Weight মাত্র ১.৬ বিট
- **Hardware** বাস্তবতা: সব কিছু বাইনারিতেই সেভ হয়, কিন্তু অনেক কম জায়গায়
- **Addition-only** গণনা: Multiplication বাদ $\rightarrow$ GPU কম লাগে $\rightarrow$ দ্রুততর

ভবিষ্যতের সম্ভাবনা

এই রিসার্চ পেপার মাত্র প্রথম ধাপ। গবেষকরা আশা করছেন যে এর বাস্তব Implementation শুরু হলে AI কমিউনিটি আমূল বদলে যাবে:

✓ সাধারণ ডিভাইসে বড় মডেল

স্মার্টফোন, ল্যাপটপে ৭০B+ মডেল চালানো সম্ভব হবে। Cloud-dependence কমবে।

✓ নতুন Hardware ডিজাইন

Ternary-optimized Chip তৈরি হলে আরও বেশি সাশ্রয় সম্ভব। Intel, AMD, Qualcomm এটি নিয়ে কাজ করতে পারে।



HuggingFace ও Implementation: গবেষকরা আশা করছেন যে HuggingFace

প্রথমে এই পদ্ধতি Implementation করবে, যাতে সাধারণ ডেভেলপাররাও সহজে BitNet মডেল ব্যবহার করে Application তৈরি করতে পারেন।

এই গাইডটি BitNet b1.58 রিসার্চ পেপার এবং সংশ্লিষ্ট শিক্ষামূলক বিষয়বস্তুর ভিত্তিতে তৈরি।

সম্পূর্ণ রিসার্চ পেপার: *"The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits"*

BitNet b1.58 — সম্পূর্ণ টেকনিক্যাল গাইড

সম্পূর্ণ বাংলায় রচিত | Technical Terms ইংরেজিতে | Python কোড সহ

একটি শিক্ষামূলক দলিল — কৃত্রিম বুদ্ধিমত্তার নতুন দিগন্ত অন্বেষণে